for machine learning. Forest is a suite of software tools for developing and running quantum applications, also developed by Rigetti Computing. It includes Pyquil and other tools for quantum programming, as well as a cloud based platform for running quantum simulations and experiments.

IBM Q Experience

https://quantum-computing.ibm.com/

IBM Q Experience is a cloud based platform for programming and running quantum circuits on IBM's quantum computers. It includes a range of tools for building and testing quantum algorithms, including quantum machine learning algorithms.

These are just some of the platforms and libraries available for quantum machine learning. As the field continues to grow, we can expect to see more tools and platforms emerge to support this exciting field of research.

Implementation scenarios

The following code demonstrates how to use a quantum circuit to classify two data points as either class 0 or class 1, based on a training data set.

```
from qiskit import QuantumCircuit, execute, Aer
from qiskit.aqua.components.feature_maps import
RawFeatureVector

# Define training data and labels
training_data = [[1, 0], [0, 1]]
training_labels = [0, 1]

# Define feature map circuit
feature_map = RawFeatureVector(feature_dimension=2, data_map_
func=lambda x: x)

# Define quantum circuit
qc = QuantumCircuit(2, 1)
qc.append(feature_map, [0, 1])
qc.h(0)
qc.cx(0, 1)

# Measure qubit 0 to obtain classification result
qc.measure(0, 0)

# Execute circuit on local simulator
backend = Aer.get_backend('qasm_simulator')
job = execute(qc, backend, shots=1024)
result = job.result()

# Print classification result
counts = result.get_counts()
print(counts)
```

In this code, we first define a training data set with two data points and their corresponding labels. We then define a feature map circuit that maps each data point to a quantum state. In this case, we use the RawFeatureVector feature map that maps each data point to a 2-qubit state.

We then define a quantum circuit with two qubits and one classical bit. We apply the feature map to the two qubits, followed by a Hadamard gate on the first qubit and a CNOT gate between the two qubits. Finally, we measure the first qubit to obtain the classification result.

We execute the circuit on a local simulator and obtain the counts of the measurement outcomes. The output will be a dictionary of measurement outcomes and their corresponding counts, such as: {'0': 483, '1': 541}.

This result indicates that the first data point was classified as belonging to class 0 with a probability of approximately 47 per cent, and belonging to class 1 with a probability of approximately 53 per cent. The actual classification depends on the threshold value used to interpret the measurement outcome.

Example of quantum machine learning for image classification

Here is an example of quantum machine learning applied to image classification using the PennyLane and TensorFlow quantum libraries:

```
import pennylane as qml
import tensorflow as tf
import tensorflow_quantum as tfq
from tensorflow.keras import layers

n_qubits = 4  # number of qubits to use in quantum
circuit
n_classes = 3  # number of classes to classify images
into

# Define a quantum circuit that will act as the classifier
dev = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev)
def circuit(inputs, weights):
    # Encoding the input data as quantum states
    for i in range(n_qubits):
        qml.RY(inputs[i], wires=i)

    # Apply the trainable weights to the circuit
    for i in range(n_qubits):
        qml.Rot(*weights[i], wires=i)

    # Measure the qubits to get the output probabilities
    return [qml.probs(wires=i) for i in range(n_qubits)]
```